FINE-TUNING & LLM OPTIMIZATION STRATEGIES

Name: _____ **Branch/Year:** _____ **Date:** _____

Warm-Up: What Do You Already Know?

1. Out-of-the-box LLMs like ChatGPT are best described as:

A. Perfect experts in every domain

B. Generalists — good at many things but master of none

C. Only capable of writing code

D. Only useful after fine-tuning

2. If you want an LLM to respond in a specific JSON format, the easiest first step is:

A. Fine-tune the model with thousands of JSON examples

B. Use a Few-Shot prompt showing the exact format you want

C. Retrain the model's internal weights

D. Change the model's temperature to 0

3. "Hallucination" in AI means:

A. The AI is dreaming

B. The AI generates confident-sounding but factually incorrect information

C. The AI refuses to answer a question

D. The AI becomes slower over time

Part 1: Why Do We Need to Optimize LLMs?

THE SCENARIO: "The Helpful But Clumsy Assistant"

Imagine you've hired a new intern. They're smart, enthusiastic, and can talk about almost anything. But when you ask them to write a customer email in your company's exact tone, they write something generic. When you ask about your internal HR policy, they *make up* an answer because they've never seen the policy document. When you need a strict JSON report, they ramble in paragraphs instead.

This is exactly how an out-of-the-box LLM behaves — it's a **generalist**. It's good at everything but a master of nothing.

To make an LLM truly useful for a specific job, we need to guide it in four key areas:

Optimization Goal	The Problem	Example
Accuracy	The AI makes up facts (hallucinations)	Inventing fake company policies
Consistency	Output format changes randomly	Sometimes JSON, sometimes plain text
Context	No access to private/domain data	Doesn't know your internal docs
Tone	Generic voice, not brand-specific	Replies like a textbook, not like your brand

ACTIVITY: Think About It:

For each goal below, write a real-world example from any company or app you use:

1. **Accuracy** matters most when:
2. **Consistency** matters most when:
3. **Tone** matters most when:

Part 2: From Prompt Engineering to Fine-Tuning

In our last lecture, we learned **Prompt Engineering** — the art of writing better instructions to get better outputs. It's powerful and solves ~80% of problems. But it has **real limitations** that Fine-Tuning can solve:

Limitation of Prompt Engineering	Why It's a Problem	How Fine-Tuning Fixes It
Temporary Instructions	Every conversation starts from scratch — the model "forgets" your instructions next time.	Fine-Tuning *permanently* bakes the behavior into the model's weights.
Inconsistent Outputs	Even with great prompts, the model sometimes ignores the format or drifts from your style.	Fine-Tuning makes the model reliably follow your format every single time.
Long, Expensive Prompts	To get consistent results, you need huge system prompts with many examples — this costs more tokens.	Fine-Tuning absorbs those examples into the model, so you need shorter prompts.
Can't Teach New Skills	No prompt can teach GPT a rare dialect, a proprietary code language, or medical terminology it's never seen.	Fine-Tuning trains the model on new data it never saw before — permanently.
Generic Personality	Prompts can nudge the tone, but the AI still sounds "generic" — not like YOUR brand.	Fine-Tuning gives the model a permanent personality, style, and voice.

THE ANALOGY

Prompt Engineering = Teaching a student for one day — tomorrow they forget everything.
Fine-Tuning = Sending the student to **medical school** for 4 years — they become a specialist *permanently*.

KEY INSIGHT: The Golden Rule: Always start with Prompt Engineering first (it's free and fast!). Fine-Tune **only** when prompting isn't enough — when you need permanent behavior, consistent style, or want to teach the model something new.

ACTIVITY: PE or FT?

For each scenario, write **PE** (Prompt Engineering) or **FT** (Fine-Tuning):

1. Making GPT summarize meeting notes in bullet points → _____
2. Training a model to perfectly copy your CEO's email writing style → _____
3. Asking the AI to translate a paragraph from English to French → _____
4. Making a small 8B model as accurate as GPT-4 at writing SQL queries → _____
5. Getting GPT to role-play as a pirate for one conversation → _____

Quick Check: What does Fine-Tuning actually change in the model?

A. It adds new documents to a database

B. It modifies the prompt template

C. The model's internal weights are updated through retraining

D. It increases the token limit

Part 3: When Should You Fine-Tune?

THE BIG QUESTION

Imagine your college wants to build an AI-powered **"Placement Prep Bot"** to help students prepare for campus placements. You need it to:

- Always reply like an **encouraging senior mentor** — motivating, supportive, with practical tips
- Format every answer in a **structured way**: Topic → Explanation → Example → Practice Question
- Use **simple language** that any student can understand (not textbook jargon)
- Always include a **motivational sign-off** like "You've got this! Keep practicing 💪 "

Could you achieve this with just a system prompt? ***Maybe for one conversation.***
But the model would keep drifting, forgetting the format, and sounding generic.
This is exactly when Fine-Tuning shines.

ACTIVITY: Think & Write:

Why would Prompt Engineering alone fail for the Placement Prep Bot? List 2 reasons:

1.
Hint: *Think: What happens after 100 conversations? Does the model stay consistent?*

2.
Hint: *Think: Can you fit the mentor personality + format rules + examples in every prompt?*

Here's a simple decision checklist — if you answer **Yes** to any of these, Fine-Tuning is worth exploring:

Decision Question	If Yes → Fine-Tune	Example
Does the model need a **permanent personality**?	Prompts can nudge tone, but it drifts. FT bakes it in.	A customer service bot that's always warm and empathetic.
Does the model need a **strict output format**?	Prompts sometimes get ignored. FT makes format reliable.	Always returning valid JSON with exactly 5 fields.
Are your prompts getting **too long and expensive**?	FT absorbs examples into weights → shorter prompts, lower cost.	A production API where each prompt costs $0.02 in tokens.
Does the model need to learn **something it's never seen**?	No prompt can teach unknown patterns. FT trains on new data.	A rare dialect, proprietary code, or domain-specific terminology.
Do you need a **small model to match a big model's** quality?	FT can specialize a cheap 8B model to rival GPT-4 on one task.	Fine-tune LLaMA-8B to match GPT-4 quality on invoice extraction.

KEY INSIGHT: The simple rule: If your problem is about *how* the AI behaves (style, format, personality, efficiency) and Prompt Engineering isn't consistent enough — that's when you Fine-Tune.

OBSERVATION: Preview — Coming in the next lecture: There's another powerful technique called **RAG (Retrieval-Augmented Generation)** that solves a *different* kind of problem — when the AI needs to know external facts (like your company's documents, latest news, etc.). We'll cover that in detail after mastering Fine-Tuning. For now, let's focus on **changing the AI's behavior permanently.**

Part 4: Fine-Tuning Deep Dive — How It Actually Works

Fine-Tuning is the process of taking a pre-trained model (like GPT-4o-mini or LLaMA) and training it further on a **smaller, specific dataset** so it permanently updates its internal weights.

In Simple Words: Imagine GPT is a general doctor. Fine-tuning is like sending that doctor to a specialized residency program — after training, they're still a doctor, but now they're an expert cardiologist. The knowledge is *baked into their brain*, not written on a sticky note.

When Should You Fine-Tune?

Use Case	Why Fine-Tuning Helps	Example
Style / Tone	Make the model sound exactly like a specific character or brand	A customer support bot that always replies with a warm, casual tone
Efficiency (Cost/Latency)	Make a small, cheap model perform as well as a huge expensive model on *one* specific task	Fine-tune LLaMA-8B to match GPT-4 quality on invoice extraction
New Skills	Teach the model something it's never seen	Training a model on a rare dialect or a proprietary programming language
Strict Format	The model must consistently output a specific structure	Always returning valid JSON with exactly 5 fields, no deviation

ACTIVITY: Fill in the Blanks:

Fine-Tuning changes the model's internal _____ .

The analogy for Fine-Tuning is: sending the student to .

Fine-Tuning is best for _____ and _____ , not for adding external facts.

The training data format for Fine-Tuning is called _____ .

Part 5: The Training Data Format — JSONL

Fine-tuning requires **example conversations** that show the AI how you want it to behave. These examples are stored in a special format called **JSONL** (JSON Lines — one JSON object per line).

In Simple Words: Think of JSONL as a recipe book for the AI. Each recipe (line) has three parts:

- **System** — Who should the AI pretend to be? (the role/persona)
- **User** — What did the customer/user say? (the input)
- **Assistant** — What is the perfect reply? (the output you want)

Example: Pizza Support Training Data

What's happening below: We create a list of example conversations, then save them as a JSONL file. Each conversation teaches the AI how a pizza support agent should reply.

```
import json

# Your training examples
# In a real scenario, you'd want 50-100 high-quality examples like these

training_data = [

    # Example 1: Customer asking about order status
    {"messages": [
        {"role": "system", "content": "You are a friendly pizza delivery support agent."},
        {"role": "user", "content": "Where is my order? I ordered 45 minutes ago."},
        {"role": "assistant", "content": "I apologize for the delay! Let me check on that right
away. Could you provide your order number?"}
    ]},

    # Example 2: Customer wants to modify an order
    {"messages": [
        {"role": "system", "content": "You are a friendly pizza delivery support agent."},
        {"role": "user", "content": "Can I add extra cheese to my pizza?"},
        {"role": "assistant", "content": "Of course! Extra cheese makes everything better. Is
there anything else you'd like to adjust on the order?"}
    ]},

    # Example 3: Handling a complaint
    {"messages": [
        {"role": "system", "content": "You are a friendly pizza delivery support agent."},
        {"role": "user", "content": "My pizza arrived cold. What can you do?"},
        {"role": "assistant", "content": "I am so sorry to hear that! That is definitely not the
experience we want for you. I can either send a fresh hot replacement right now or issue a full
credit to your account. Which would you prefer?"}
    ]},

    # Example 4: Dietary question
    {"messages": [
        {"role": "system", "content": "You are a friendly pizza delivery support agent."},
        {"role": "user", "content": "Do you have any gluten-free crust options?"},
        {"role": "assistant", "content": "We certainly do! We offer a cauliflower-based gluten-
free crust for any of our medium-sized pizzas."}
    ]},

    # Example 5: Promotions
    {"messages": [
        {"role": "system", "content": "You are a friendly pizza delivery support agent."},
        {"role": "user", "content": "Do you have any buy-one-get-one deals today?"},
        {"role": "assistant", "content": "Today is 'Two-for-Tuesday'! If you order any large
specialty pizza, you get a second one of equal value for free. Would you like to add that to
your cart?"}
    ]},

]

# Write the data to a JSONL file
# Each JSON object goes on its own line — this is the format AI providers expect

with open("pizza_support_training_data.jsonl", "w") as f:
    for item in training_data:
        f.write(json.dumps(item) + "\n")

print("Training data saved to pizza_support_training_data.jsonl!")
print("Next step: Upload this file to your AI provider to start training.")
```

OBSERVATION: Notice the pattern: every single training example follows the same **System → User → Assistant** structure. Consistency is key! The AI learns the *pattern* of how to respond, not just the specific answers.

ACTIVITY: Write Your Own Training Example:

Imagine you're fine-tuning a model to be a **hotel booking assistant**. Write one JSONL training example with all three roles:

YOUR JSONL EXAMPLE (SYSTEM, USER, ASSISTANT):

Part 6: Path A — Fine-Tuning with OpenAI API (Managed)

This is the **simplest way** to fine-tune. OpenAI handles all the heavy lifting — servers, GPUs, training loops — you just upload your training data and wait.

In Simple Words: Think of it like dropping your clothes at a professional dry cleaner. You hand over the clothes (data), they clean them (train the model), and you pick up the results. You don't need to own a washing machine (GPU)!

Step 1: Upload Your Data & Start Training

What's happening below: We connect to OpenAI, upload our JSONL training file, and tell OpenAI to start training a custom version of GPT-4o-mini.

```
import os
from openai import OpenAI

# Connect to OpenAI using your secret API key
# NEVER hardcode your key — always use environment variables!
client = OpenAI(api_key=os.getenv("OPENAI_KEY"))


# Step 1: Upload the training examples file to OpenAI
training_file = client.files.create(
    file=open("pizza_support_training_data.jsonl", "rb"),   # Your JSONL file
    purpose="fine-tune"                                      # Tell OpenAI it's for training
)

print(f"File uploaded! ID: {training_file.id}")


# Step 2: Start the fine-tuning job
# This can take anywhere from a few minutes to a few hours
job = client.fine_tuning.jobs.create(
    training_file=training_file.id,       # Use the file we just uploaded
    model="gpt-4o-mini-2024-07-18"        # The base model to customize
)

print(f"Training started! Job ID: {job.id}")
```

Step 2: Check If Training Is Done

What's happening below: Training takes time. We check the status of our job to see if it's finished.

```
# Check the status of your training job
job = client.fine_tuning.jobs.retrieve("ftjob-YOUR_JOB_ID_HERE")

if job.status == "succeeded":
    print(f"Training complete! Your custom model ID: {job.fine_tuned_model}")
else:
    print(f"Current Status: {job.status}. Please wait a few more minutes.")
```

Step 3: Use Your Custom Model

What's happening below: After training, you get a custom model ID. You use it exactly like the regular OpenAI API — just swap the model name!

```
# Replace this with YOUR custom model ID from Step 2
FINE_TUNED_MODEL = "ft:gpt-4o-mini-2024-07-18:personal::YOUR_MODEL_ID"

# Use it exactly like before — same API, just a different model name!
response = client.chat.completions.create(
    model=FINE_TUNED_MODEL,
    messages=[
        {"role": "system", "content": "You are a friendly pizza delivery support agent."},
        {"role": "user", "content": "Where is my pepperoni pie?"}
    ],
    temperature=0.7     # Controls creativity (0 = strict, 1 = creative)
)

# Print the AI's reply
reply = response.choices[0].message.content
print(f"Agent: {reply}")
```

WARNING: **Never hardcode API keys in your code!** Always use environment variables: `os.getenv("OPENAI_KEY")` . Hardcoded keys in public repos get stolen within minutes.

Part 7: Path B — Open-Source Fine-Tuning with LoRA

If you need **full privacy** (your data never leaves your computer) or want **total control** over the training process, you can fine-tune an open-source model yourself.

In Simple Words: Instead of sending clothes to a dry cleaner (OpenAI), you buy your own washing machine (GPU) and do everything at home. More work, but total privacy and control.

What is LoRA?

Normally, fine-tuning a 7-billion parameter model requires 32GB+ of GPU memory — that's a $1,500+ graphics card! **LoRA (Low-Rank Adaptation)** is a clever trick that solves this problem.

In Simple Words: Instead of retraining the entire AI brain (which is expensive), LoRA **freezes 99% of the model** and only trains tiny "adapter" layers on top. Think of it as putting a specialized hat on the AI instead of rebuilding its whole head!

OBSERVATION: With LoRA, you can fine-tune a 1B parameter model with just ~4GB of GPU memory — that's available for free on Google Colab!

Step 1: Install Libraries & Load Model

What's happening below: We install the required libraries, load a small free model (TinyLlama), and attach LoRA adapters to it.

```
# Install the required libraries (run this once)
!pip install -q trl peft datasets transformers

from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import get_peft_model, LoraConfig


# Load a small, free model (TinyLlama — 1.1 billion parameters)
model_id = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"

tokenizer = AutoTokenizer.from_pretrained(model_id)
tokenizer.pad_token = tokenizer.eos_token     # TinyLlama needs this for batching

model = AutoModelForCausalLM.from_pretrained(model_id)


# Attach LoRA adapters — this is THE memory-saving trick!
# r=8 means the adapter is very small (rank 8)
# target_modules tells LoRA WHICH layers to adapt
peft_config = LoraConfig(r=8, target_modules=["q_proj", "v_proj"])

model = get_peft_model(model, peft_config)
```

Step 2: Prepare Data & Train

What's happening below: We load the same JSONL file we created earlier, format it for training, and run the training loop.

```
 from datasets import load_dataset
from transformers import TrainingArguments, Trainer


# Load our pizza support training data
dataset = load_dataset("json", data_files="pizza_support_training_data.jsonl")


# Format the data: apply the chat template and tokenize
def format_data(examples):
    texts = [tokenizer.apply_chat_template(msg, tokenize=False)
             for msg in examples["messages"]]

    inputs = tokenizer(texts, padding="max_length",
                       max_length=128, truncation=True)

    # Tell the model to calculate loss against its own inputs
    inputs["labels"] = inputs["input_ids"]
    return inputs

processed_dataset = dataset.map(format_data, batched=True,
                                remove_columns=["messages"])


# Configure and start training
args = TrainingArguments(
    output_dir="./tmp",           # Where to save checkpoints
    num_train_epochs=50           # How many times to loop through the data
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=processed_dataset["train"],
)

# This is where the magic happens — the model learns!
trainer.train()

# Save the trained adapter weights (NOT the full model — just the tiny LoRA part)
trainer.save_model("./pizza-support-model")
print("Training complete! Model saved locally.")
```

Step 3: Use Your Locally Trained Model

What's happening below: We load the base model, attach our trained LoRA adapter, merge them together, and chat with our custom assistant!

```
 from peft import PeftModel
from transformers import pipeline


# Load the base model again
base = AutoModelForCausalLM.from_pretrained(model_id)

# Attach our trained LoRA adapter and merge it into the base
p_model = PeftModel.from_pretrained(
    base, "./pizza-support-model"
).merge_and_unload()


# Create a simple chatbot pipeline
bot = pipeline("text-generation", model=p_model, tokenizer=tokenizer)

print("Local Pizza Agent (Type 'exit' to quit)")
while True:
    user_input = input("Customer: ")
    if user_input.lower() in ['exit', 'quit']:
        break

    reply = bot(f"Customer: {user_input}", max_new_tokens=50)
    print(f"Agent: {reply[0]['generated_text']}\n")
```

Quick Check: What is the main advantage of LoRA over full fine-tuning?

A. It trains faster because it skips validation

B. It uses dramatically less GPU memory by only training tiny adapter layers

C. It produces better results every time

D. It doesn't require any training data

Part 8: Limitations of Fine-Tuning

Fine-tuning is powerful, but it is **not a magic wand**. Understanding its limitations is just as important as knowing when to use it.

Limitation	Explanation	What to Do Instead
Does NOT Fix Hallucinations	Fine-tuning on incorrect or insufficient data can actually *increase* hallucinations. The model memorizes wrong patterns.	Use **good prompting** to ground answers, or wait for our next lecture on a technique designed exactly for this.
Needs High-Quality Data	The output is only as good as the training data. Garbage in = garbage out. 50 excellent examples beat 1,000 bad ones.	Invest time in curating data, not collecting more of it.
Expensive & Slow to Update	Every time your data changes, you need to retrain. This costs money and time.	Only fine-tune for **stable behaviors** that don't change often.
Catastrophic Forgetting	Training too hard on a narrow dataset can make the model *forget* its general abilities.	Use LoRA adapters (they preserve the base model) and moderate epochs.
Frozen Knowledge	A fine-tuned model only knows what was in its training data — it can't look up today's stock price.	Fine-Tuning teaches *behavior*, not live facts. For live data, we'll learn another technique next lecture.

WARNING: **The #1 Myth:** "Fine-tuning will fix my AI's hallucination problem."
Reality: If the AI is making up facts, fine-tuning won't help — it can actually make hallucinations *worse* by memorizing incorrect patterns. Fine-tuning is for **behavior**, not for teaching the AI new facts.

ACTIVITY: True or False:

1. Fine-tuning is the best way to fix hallucinations. _____

2. 50 high-quality training examples are better than 1,000 bad ones. _____

3. Once fine-tuned, the model automatically stays up-to-date. _____

4. LoRA helps prevent catastrophic forgetting. _____

Part 9: Prompt Engineering vs. Fine-Tuning — Side by Side

Let's put everything together. Here's a comprehensive comparison of the two techniques you've learned so far:

Dimension	Prompt Engineering	Fine-Tuning
How It Works	You write instructions in the prompt each time	The AI's internal weights are permanently retrained
Persistence	Temporary — resets every conversation	Permanent — baked into the model forever
Consistency	Model sometimes drifts or ignores instructions	Highly consistent — format and style are reliable
Setup Cost	Free — just write the prompt	Expensive — GPU/API costs + training time
Token Cost	High — long prompts with examples eat tokens	Lower — behavior is internalized, shorter prompts needed
Can Teach New Skills?	No — can only use what the model already knows	Yes — can learn new dialects, code languages, terminology
Latency	Slightly slower (longer prompts)	Faster at inference (knowledge is baked in)
Best For	Quick tasks, one-off formatting, exploration	Permanent style, strict formats, specialized behavior, efficiency

KEY INSIGHT: **The progression:** Prompt Engineering is where you *start*. Fine-Tuning is where you go when prompting alone isn't reliable enough for production use. Think of it as: Prompt Engineering = prototyping, Fine-Tuning = production.

ACTIVITY: PE or FT?

For each scenario, write **PE** or **FT**:

1. A legal chatbot that must always output exactly the same JSON structure → _____

2. A customer service bot that always uses a warm, empathetic tone → _____

3. Quickly test if GPT can summarize your meeting notes → _____

4. Making a small model (8B params) as good as GPT-4 at writing SQL queries → _____

5. Asking GPT to explain recursion with one specific analogy → _____

Part 10: The Decision Flowchart

Here's your **complete comparison** of Prompt Engineering vs Fine-Tuning:

Feature	Prompt Engineering	Fine-Tuning
Setup Time	Minutes	Hours to Days
Cost	Low (just API calls)	High (GPU or API training costs)
Best For	Quick tasks, formatting, simple instructions	Style, tone, strict format, efficiency, new skills
Accuracy	Good for simple tasks	Excellent for specialized behavioral tasks
Memory Needed	None (context window only)	GPU VRAM for training
Maintenance	Low — just update the prompt	High — must retrain on new data

THE DECISION FLOWCHART

Step 1: Can Prompt Engineering solve it? → If yes, **stop here!**
↓ (If no...)
Step 2: Is the problem about *behavior* (style, format, personality)? → **Fine-Tune!**
↓ (If the problem is about *facts or live data*...)
Step 3: Stay tuned for our next lecture! We'll learn a technique designed exactly for this.

A startup wants their AI to always respond in a friendly, casual tone AND follow a strict JSON output format. What should they use?

A. Only Prompt Engineering

B. Fine-Tuning (for permanent tone + format)

C. Just increase the temperature

D. Use a bigger model

Part 11: Common Myths & Pitfalls

ACTIVITY: Myth Buster — Circle True or False:

1. "Fine-tuning fixes hallucinations."
True / False — Because:

2. "You need millions of training examples to fine-tune."
True / False — Because:

3. "Fine-tuning makes the model smarter at everything."
True / False — Because:

4. "Prompt Engineering is always enough if you write good enough prompts."
True / False — Because:

Key Takeaways on Pitfalls:

- **Myth:** "Fine-tuning fixes hallucinations." → **Reality:** Fine-tuning teaches *behavior*, not facts. Bad training data makes hallucinations *worse*.
- **Myth:** "More data is always better." → **Reality:** Quality > Quantity. 50–100 high-quality, well-formatted examples are better than 1,000 sloppy ones.
- **Myth:** "Fine-tuning makes the model smarter at everything." → **Reality:** It makes the model better at *one specific task* but can cause catastrophic forgetting of general abilities.

KEY INSIGHT: Remember the golden rule: **Prompt Engineering first, Fine-Tuning second.** In our next lecture, you'll learn a third powerful technique that completes the toolkit — designed specifically for when the AI needs access to external knowledge.

Part 12: Hardware & Key Terminology

GPU Memory Requirements for Fine-Tuning

Model Size	Full Fine-Tuning VRAM	LoRA VRAM	Recommended GPU
1B (TinyLlama)	~8GB	~4GB	NVIDIA T4 (Free Colab)
7B (LLaMA-3)	~32GB+	~14GB	RTX 3090 / 4090 / A100
70B (LLaMA-3 Large)	~280GB+	~48GB	Multiple A100s / H100s

ACTIVITY: Terminology Matching:

Match each term to its definition (draw lines or write letters next to numbers):

1. JSONL
2. LoRA
3. Epoch
4. VRAM
5. Catastrophic Forgetting

A. Memory-efficient fine-tuning technique that trains only small adapters
B. One JSON object per line — the training data format
C. Video RAM — GPU memory needed for training
D. One full pass through the entire training dataset
E. When fine-tuning makes the model forget its general abilities

1:__ 2:__ 3:__ 4:__ 5:__

Part 13: Final Challenge — The AI Architect

THE GRAND CHALLENGE

A **Law Firm** comes to you with this problem: They need to pull out dates and money amounts from 5,000 contracts every single day. They need very high accuracy and absolutely zero data leaks (contracts are top-secret and confidential!).

Using everything you learned today, design their AI system:

1. Which Path (A: OpenAI API or B: LoRA) would you pick for maximum privacy? _____
   Hint: *Think: Which path keeps data on their own computers?*
2. Why is Fine-Tuning the right approach for this task? Give 2 reasons:

YOUR JUSTIFICATION:

3. Write a System Prompt for this legal AI assistant:

YOUR SYSTEM PROMPT:

4. What output format should the AI use? _____

   Hint: *Think: Which format is structured and machine-readable?*

5. Write ONE JSONL training example for this legal extraction task:

YOUR JSONL EXAMPLE:

Recap: Your Learning Map

- **Prompt Engineering limitations:** Temporary, inconsistent, can't teach new skills.
- **Fine-Tuning** solves these by permanently retraining the model's weights.
- **When to Fine-Tune:** Permanent style, strict format, new skills, cost efficiency.
- **JSONL** is the training data format — System/User/Assistant triplets.
- **Path A (OpenAI):** Easiest setup — upload data, wait, use the custom model.
- **Path B (LoRA):** Full privacy — trains tiny adapters on your own GPU, saves ~90% memory.
- **Limitations:** Fine-tuning does NOT fix hallucinations. Quality > Quantity for data.
- **Next Lecture:** We'll learn **RAG** — the technique for giving AI access to external knowledge and live data.